
INTELIGENCIA ARTIFICIAL EMBEBIDA CLASIFICADOR DE TERRENO

Clasificación de Terreno y Análisis Vibracional mediante Nano Edge AI

Autor: Pedro Lacárcel Jardi

Tutor: Juan Pedro Domínguez

Grado: Ingeniería Informática – Ingeniería de Computadores

Centro: Escuela Técnica Superior de Ingeniería Informática

Universidad: Universidad de Sevilla

Fecha: mayo 2026

BLOQUE 1: DESCRIPCIÓN DEL PROYECTO.....	3
B1.1. Introducción y Motivación	3
Una Nueva Era.....	3
Tipos de IA y sus ámbitos de aplicación	3
1. IA Centralizada (Cloud-based AI).....	4
2. Machine Learning Tradicional (Estadístico).....	4
3. Deep Learning (Redes Neuronales)	4
4. TinyML (IA Empotrada)	4
¿Cómo funciona el TinyML?	4
El TinyML vs Programas clásicos, por qué usar IA Empotrada	5
Monitorización de temperatura en un disipador.....	5
Detector de anomalías de un motor/ventilador	6
Cómo actúa la IA en estos casos.....	6
B1.2. Objetivos del Proyecto.....	6
Objetivos profesionales	6
Objetivos educacionales	7
B1.3. Estado del Arte.....	7
Bosch Rexroth - ODiN (Online Diagnostics Network).....	7
Seeed Studio - Ecosistema Wio Terminal y Grove Vision AI	9
B1.4. Elicitación de Requisitos y Análisis de Riesgos	10
Requisitos Hardware (RH)	10
BLOQUE 2: EJECUCIÓN DEL PROYECTO.....	12
B2.1. Diseño del Sistema	12
Esquema General y Diagrama de Bloques.....	12
B2.2. Implementación.....	12
Tecnologías Empleadas.....	12
Hardware	12
Software	13
Desarrollo.....	13
Fase 1: Capa de Adquisición (Proyecto STM32) – Data logger	13
Configuración STMCubeIDE (DataLogger.ioc)	14

Fase 2: Capa de Red (Puente ESP32)	15
Fase 3: Capa de IA (Pipeline de Datos)	15
BLOQUE 3: PLANIFICACIÓN DEL PROYECTO	16
B3.2. Planificación Financiera Inicial	16
VII. CONCLUSIONES Y TRABAJO FUTURO	16

BLOQUE 1: DESCRIPCIÓN DEL PROYECTO

B1.1. Introducción y Motivación

Una Nueva Era

En la actualidad, nos hallamos ante un hito disruptivo en la evolución tecnológica: la convergencia de la Inteligencia Artificial (IA) con prácticamente todos los estratos de la cotidianidad y la industria. En un entorno globalizado y digitalizado, resulta complejo identificar dispositivos que carezcan de algoritmos inteligentes. No obstante, el despliegue de la IA en los **sistemas empotrados (embedded systems)** plantea un desafío técnico significativo. Tradicionalmente, el aprendizaje profundo y el procesamiento de grandes volúmenes de datos (*Big Data*) han requerido infraestructuras computacionales robustas; sin embargo, los sistemas embebidos operan bajo restricciones críticas de **consumo energético, capacidad de memoria y latencia de respuesta**.

Los sistemas de propósito específico, a menudo sujetos a determinismo en **tiempo real duro**, priorizan la eficiencia sobre la complejidad algorítmica. En aplicaciones críticas, como los **sistemas de frenado antibloqueo (ABS)** o el control de estabilidad vehicular, el procesamiento debe realizarse en escalas de microsegundos, donde un retardo computacional excesivo comprometería la seguridad del sistema o un mal funcionamiento de este. Por ello, a priori, cabe pensar que la IA no es buena aliada de los sistemas embebidos en tiempo real duro, donde los sistemas (microcontroladores, procesadores...) son de recursos limitados, priorizando el consumo energético y el precio. Por ello, puede existir la percepción de que la IA —debido a su alta carga computacional— es incompatible con microcontroladores de baja potencia y frecuencias de reloj reducidas.

A pesar de estas limitaciones, la emergencia del **TinyML** y la optimización de modelos han demostrado que es posible implementar IA de manera eficiente en un microcontrolador con recursos limitados. Por todo esto debemos dejar claro el posible uso de la IA, que hoy en día es una palabra que se usa en muchísimos ámbitos y es altamente escuchada y emitida por todos nosotros. Para usar esta tecnología tan emergente debemos de tener en cuenta los “tipos de IA” que existen y para que se utiliza cada uno.

Tipos de IA y sus ámbitos de aplicación

A menudo, al hablar de Inteligencia Artificial (IA), el imaginario colectivo se dirige automáticamente hacia los grandes modelos de lenguaje (LLMs) o sistemas generativos que requieren infraestructuras de computación masiva en la nube. Sin embargo, es fundamental aclarar que **intentar ejecutar este tipo de IA generativa en microcontroladores de recursos**

limitados, como los basados en arquitecturas **ESP32 o STM32**, resulta técnicamente inviable. Estos dispositivos carecen de la capacidad de memoria, la frecuencia de reloj y el paralelismo necesario para gestionar la carga computacional de dichos modelos.

No obstante, existe un paradigma paralelo y de gran relevancia actual: la **IA Empotrada (TinyML)**. Esta disciplina permite trasladar la inteligencia al "borde" (*edge*), permitiendo que los dispositivos de bajo consumo y recursos limitados realicen inferencias locales sin depender de la conectividad constante a la nube.

1. IA Centralizada (Cloud-based AI)

Son los modelos más complejos (como GPT o modelos de visión por ordenador pesados). Se ejecutan en servidores potentes con GPUs dedicadas. Su principal ventaja es la precisión y versatilidad, pero dependen críticamente de la latencia de red y plantean preocupaciones sobre la privacidad de los datos.

2. Machine Learning Tradicional (Estadístico)

Utiliza algoritmos clásicos como **Regresión Lineal, Árboles de Decisión o K-Means**. Son extremadamente eficientes y pueden ejecutarse en microcontroladores, pero suelen tener dificultades cuando los datos de entrada son complejos o no estructurados (como audio o imágenes crudas).

3. Deep Learning (Redes Neuronales)

Se basa en arquitecturas de capas múltiples para extraer características complejas de los datos. Aunque el entrenamiento suele requerir servidores potentes, el despliegue del modelo ya entrenado (*inferencia*) puede optimizarse considerablemente para arquitecturas más pequeñas.

4. TinyML (IA Empotrada)

Es la vanguardia de la computación eficiente. **TinyML es una subdisciplina del Machine Learning que se enfoca en el despliegue de modelos de aprendizaje profundo en dispositivos embebidos de muy bajo consumo.**

¿Cómo funciona el TinyML?

El proceso no consiste en "entrenar" en el dispositivo, sino en un ciclo de optimización:

1. **Entrenamiento:** Se entrena una red neuronal en un entorno potente (PC o Cloud) con grandes volúmenes de datos.
2. **Optimización:** Se aplican técnicas de **cuantización** (reducir la precisión de los pesos de la red, pasando de 32 bits a 8 bits) y **poda** (*pruning*, eliminando conexiones innecesarias en la red) para reducir el tamaño del modelo.

3. **Conversión:** El modelo se traduce a un formato legible por el microcontrolador.
4. **Inferencia:** El microcontrolador ejecuta el modelo optimizado utilizando únicamente su memoria RAM y Flash local, permitiendo la detección de anomalías o clasificación de estados en milisegundos y con un consumo energético ínfimo.

Este enfoque es el que implementaré en este TFG, priorizando la eficiencia algorítmica para lograr una detección de anomalías robusta, capaz de operar de manera autónoma en tiempo real.

El TinyML vs Programas clásicos, por qué usar IA Empotrada

Para entender el potencial de esta tecnología me ha parecido interesante comparar la facilidad de desarrollo e implementación entre ambas, con el caso más extendido de uso: las **vibraciones**.

Cuando intentamos detectar anomalías en vibraciones —como un rodamiento desgastándose— o sonidos complejos, como un motor que empieza a chirriar, nos enfrentamos a un problema de **indeterminación**. A diferencia de medir una temperatura (donde existe un umbral crítico de grados Celsius) o detectar una señal de radio para abrir una puerta, aquí no hay un valor único o un patrón claro que sea fácil de interpretar mediante lógica booleana.

En los casos de temperatura o velocidad, el uso de reglas deterministas es la solución óptima: basta con un simple if-else para obtener un resultado robusto y rápido. Intentar implementar IA para resolver tareas tan triviales sería el equivalente tecnológico a “ir a comprar el pan en un tanque de guerra”, estaríamos consumiendo recursos de cómputo, energía y memoria de forma desproporcionada, complicando innecesariamente un sistema que ya funciona de manera eficiente.

Pero ¿qué ocurre si estamos detectando las vibraciones de una tubería, un motor o un ventilador que está en mal estado?, ¿cómo determinamos cuándo está en mal estado?, ¿qué valor le pasamos al if-else? En principio, son preguntas difíciles de responder. Puede un motor que vibre mucho pero que sea de un acelerón (comportamiento normal), o que su vibración sea menos abrupta porque está en reposo. El desarrollo de esa detección se puede volver muy complejo, ya que no existe un valor determinado de esa vibración que indique “mal funcionamiento”. Puede que en algún momento de la vida del motor empiece a sonar un pequeño silbido que indique una futura avería, como lo identificamos. Veamos ejemplos:

Monitorización de temperatura en un disipador

```
if (temperatura > 85.0) {  
    encender_ventilador_emergencia();  
    enviar_alerta("Sobrecalentamiento");  
}
```

La temperatura es un dato determinado que podemos manejar sin problemas y con un programa de este tipo podemos dar una solución gastando recursos mínimos.

Detector de anomalías de un motor/ventilador

Pero como manejamos las vibraciones de un motor o ventilador para detectar un funcionamiento inusual, una solución sencilla podría ser:

```
if (vibracion_x > 0.5 || vibracion_y > 0.8) {  
    lanzar_alarma();  
}
```

En este caso estaríamos determinando que si supera un valor en nuestro acelerómetro o sensor lanzamos una alarma. Tendríamos un 50% de acertar. Pero ¿y si ese valor lo supera por un acelerón o por arrancar?

Es un sistema multifactorial, donde se puede superar o no ciertos valores y no siempre acertará. Para determinar si de verdad es un fallo tardaríamos mucho en desarrollar un algoritmo certero que determine cuando es un fallo, estableciendo un flujo valorando factores que puede llegar a ser incluso costoso y con una respuesta lenta, que puede afectar al tiempo real duro que debemos tener en cuenta en sistemas de esta naturaleza.

Cómo actúa la IA en estos casos

Veamos como un **clasificador de anomalías con TinyML** marca la diferencia. La IA no busca un valor, busca un **patrón de comportamiento**. Mediante el uso de técnicas de *One-Class Classification* (usando NanoEdge AI Studio), el sistema aprende cómo "se siente" el motor cuando está sano, incluyendo todas sus variantes de uso normales (arranque, reposo, carga máxima).

Ejemplo con pseudocódigo:

```
float umbral_anomalia = NanoEdgeAI_run(buffer_acelerometro);  
  
if (umbral_anomalia > 0.9) {  
    activar_mantenimiento_preventivo();  
}
```

En este caso está valorando “cuanto” se parece ese flujo de datos que recoge el acelerómetro al comportamiento normal, si es más de un 90% parecido a lo que “entiende” por anomalía lanza la alarma, de esta forma la implementación es altamente sencilla, dando paso a una amplia gama de casos de uso en sistemas modernos.

B1.2. Objetivos del Proyecto

Objetivos profesionales

- **Implementación de un Pipeline de Inferencia Edge-AI:** Desarrollar un flujo de trabajo completo que abarque desde la captura de señales inerciales de alta frecuencia, pasando por el preprocesamiento de señales, hasta la ejecución de una librería de inferencia optimizada mediante **NanoEdge AI Studio**.
- **Optimización de Algoritmos en Hardware de Bajo Recurso:** Demostrar la viabilidad de ejecutar algoritmos de clasificación de estados (como *Anomaly Detection* y *Classification*) en arquitecturas basadas en **STM32/ESP32**, logrando un balance óptimo entre precisión diagnóstica y consumo de recursos (RAM < 100KB y uso eficiente de ciclos de reloj).

- **Desarrollo de un Sistema de Monitorización Robusto:** Diseñar una arquitectura de software que garantice el determinismo en procesos críticos, empleando sistemas de telemetría híbridos (UART/WiFi) para la visualización remota de datos sin comprometer la estabilidad del lazo de control principal del vehículo/motor.
- **Validación de Metodologías de Detección:** Comparar cuantitativamente la eficacia de un detector de anomalías basado en IA (aprendizaje de firmas espectrales) frente a la lógica determinista convencional, midiendo métricas como la tasa de falsos positivos y el tiempo de respuesta ante estados transitorios.

Objetivos educativos

- **Evaluación de la Eficiencia en AutoML:** Analizar las ventajas operativas y la reducción en el tiempo de desarrollo que ofrecen las herramientas de *AutoML* (NanoEdge AI Studio) en comparación con el desarrollo manual (extracción de características mediante filtros manuales y FFT).
- **Clasificación Adaptativa de Entornos:** Implementar un clasificador capaz de identificar distintos estados operacionales (ej. tipos de terreno o estados de carga del motor) mediante el procesamiento de patrones de vibración.
- **Integración de Sistemas Embebidos:** Profundizar en la interacción entre capas de abstracción de hardware (**STM32 HAL / RTOS**) y modelos de IA pre-compilados, garantizando la coexistencia armoniosa entre el sistema operativo de tiempo real y la carga computacional de la inferencia.

B1.3. Estado del Arte

Bosch Rexroth - ODiN (Online Diagnostics Network)

El sistema **ODiN** de Bosch Rexroth es una solución avanzada de analítica predictiva diseñada para monitorizar el estado de maquinaria industrial, especialmente en sectores donde los costes de inactividad son críticos, como la metalurgia o la industria papelera.

ODiN funciona como un servicio basado en la nube. Recopila datos de sensores (vibración, presión, temperatura, calidad del aceite) a través de un *IoT Gateway* que transmite esta información a un centro de datos de Bosch. Allí, algoritmos de Machine Learning analizan los patrones de comportamiento para identificar desviaciones que los sistemas de umbrales fijos serían incapaces de detectar.

Pros:

- **Fiabilidad Industrial:** Es un sistema probado para evitar paradas inesperadas en entornos de producción masiva.

- **Análisis Multivariable:** No solo se basa en una variable, sino que correlaciona datos de hasta 2000 *endpoints* distintos para dar un diagnóstico preciso.
- **Servicio Integral:** Bosch no solo vende el hardware, sino que aporta el conocimiento experto ("domain knowledge") para interpretar las anomalías.

Contras:

- **Arquitectura de "Caja Negra" (Cloud-Dependent):** Depende totalmente de la nube para el procesamiento. Si la conexión de red falla o el servicio del fabricante se interrumpe, el sistema pierde su capacidad analítica.
- **Latencia elevada:** Al procesar en servidores remotos, no es apto para aplicaciones de **tiempo real duro**.
- **Coste y Complejidad:** Requiere una inversión elevada en infraestructura y licencias, flotas de robots móviles o dispositivos de bajo coste.

https://2022461.fs1.hubspotusercontent-na1.net/hubfs/2022461/DC-IH_MKT/Software-and-Tools/CytroConnectSolutions/01_Whitepaper1_CCS_PdM_EN.pdf

<https://www.boschrexroth.com/en/hu/service-and-support/service/predictive-maintenance/>

Si bien las soluciones comerciales actuales ofrecen una robustez indiscutible en entornos de planta, su dependencia de infraestructuras basadas en la nube y su naturaleza como 'caja negra' limitan drásticamente su aplicabilidad en sistemas autónomos y dispositivos móviles.

La aportación fundamental de mi trabajo radica en el traslado de la capacidad analítica desde el servidor remoto hasta el propio nodo de procesamiento (*Edge Computing*). Mientras que los sistemas convencionales exigen una conectividad constante y el procesamiento masivo de datos en centros externos, mi propuesta implementa la inferencia local mediante **TinyML y NanoEdge AI Studio**. Esto conlleva las siguientes ventajas:

- **Autonomía y Determinismo:** Al ejecutar el modelo directamente sobre el microcontrolador (STM32), el sistema elimina la latencia de red y la dependencia de un tercero. Esto es crítico en aplicaciones de tiempo real, donde una decisión de mantenimiento predictivo debe tomarse en milisegundos para evitar una falla catastrófica, sin estar condicionados por la disponibilidad de una conexión inalámbrica.
- **Eficiencia Energética y de Costes:** Mi enfoque democratiza el mantenimiento predictivo. A diferencia de las soluciones industriales de alta gama que requieren una inversión significativa y licencias de suscripción, mi solución demuestra que es posible desplegar algoritmos de detección de anomalías de grado industrial en hardware de bajo coste, optimizando el consumo energético.

Seeed Studio - Ecosistema Wio Terminal y Grove Vision AI

El ecosistema de Seeed Studio representa la vertiente más accesible y orientada al aprendizaje del *Edge AI*. Wio Terminal es una placa de desarrollo basada en el microcontrolador ATSAMD51 (ARM Cortex-M4F), mientras que el Grove Vision AI V2 es un módulo especializado con procesador Himax (Cortex-M55 + NPU Ethos-U55) para tareas de visión artificial.

Estos dispositivos están diseñados bajo una filosofía *plug-and-play*. Permiten integrar sensores, pantallas y cámaras mediante conectores estandarizados, facilitando la ejecución de modelos de aprendizaje automático mediante marcos de trabajo como TensorFlow Lite for Microcontrollers o Edge Impulse. Están orientados a la validación rápida de conceptos y la educación tecnológica.

Pros:

- **Curva de aprendizaje acelerada:** Es la puerta de entrada ideal para desarrolladores y estudiantes que buscan resultados inmediatos sin profundizar excesivamente en la arquitectura de bajo nivel.
- **Accesibilidad:** Su ecosistema de módulos Grove y la integración con herramientas como Edge Impulse permiten muestrear datos y desplegar modelos sin necesidad de programar toda la cadena de adquisición.
- **Versatilidad de prototipado:** Permiten integrar múltiples modalidades sensoriales (audio, imagen, inercia) en un solo dispositivo portátil.

Contras (La brecha hacia la industria):

- **Ausencia de rigor industrial:** Aunque son herramientas potentes, carecen de las certificaciones, la robustez térmica y la fiabilidad necesaria para entornos industriales críticos.
- **Limitaciones en modelos de mantenimiento predictivo:** Los modelos entrenados en estos entornos suelen tener dificultades para generalizar en condiciones de ruido real no controladas. A menudo, las soluciones son más "demostrativas" que robustas, fallando al intentar llevar el prototipo a una implementación de alta precisión en maquinaria pesada.
- **Desacople de la arquitectura:** Al depender de librerías de terceros muy genéricas, es difícil realizar una optimización de bajo nivel sobre el consumo energético o la latencia extrema, aspectos vitales en sistemas embebidos de grado profesional.

A diferencia de la filosofía *Maker* de Seeed Studio, este proyecto busca la **eficiencia absoluta del firmware**. Mientras que las soluciones de Seeed priorizan la facilidad de uso del usuario, mi propuesta prioriza el **rendimiento del recurso**. Al utilizar un flujo de trabajo basado en **NanoEdge AI Studio** sobre arquitectura STM32.

https://wiki.seeedstudio.com/es/tinymml_topic/

B1.4. Elicitación de Requisitos y Análisis de Riesgos

Requisitos Hardware

El sistema se fundamenta en una arquitectura de procesamiento distribuido donde la capacidad computacional de IA reside en un nodo de control, mientras que las tareas de conectividad se segregan para garantizar la integridad de los procesos de tiempo real.

1. **MICROCONTROLADOR - STM32 NUCLEO L476RG:** Microcontrolador basado en ARM Cortex-M4F (80 MHz). Su FPU y capacidad de gestión de periféricos DMA lo hacen ideal para el procesamiento de señales (FFT) y la ejecución de la librería de inferencia NanoEdge. Además, cuenta con sistema operativo FreeRTOS lo que permite el manejo sencillo y controlado de tareas. Se tomará como el módulo central del proyecto, el cual correrá el modelo TinyML elegido y será el máximo responsable en el tratamiento de los datos. Destaca por su bajo coste, aunque carece de sensores integrados, no como en placas de gama superior dentro del fabricante.
2. **Acelerómetro - Sensor MPU6050:** Unidad de Medición Inercial (IMU) de 6 ejes (acelerómetro y giroscopio). Proporciona los datos cinemáticos en crudo necesarios para alimentar el clasificador de anomalías. Debido a la escasez de sensores en el microcontrolador central me he visto obligado a integrarlo, lo que ha requerido un montaje hardware adicional, el cual no contemplaba en un principio. No obstante, al ser un módulo independiente permite la instalación y colocación libre de este.
3. **Módulo ESP-WROOM-32:** Periférico dedicado exclusivamente a la telemetría y comunicación WiFi. Su rol es actuar como pasarela (bridge) hacia la red, evitando que las latencias del stack TCP/IP afecten al STM32. Destaca por su bajo coste y módulos de comunicación inalámbrica integrados, no obstante, la calidad de construcción y materiales es cuestionable. Debido a esto, estará dedicado exclusivamente al envío de datos vía WiFi desde el microcontrolador central hacia el PC/servidor, restándole así responsabilidad en el manejo y procesamiento de los datos, reduciendo los posibles errores.

Análisis de Riesgos y Decisiones de Diseño Hardware

1. **Riesgo de Interferencias por Latencia en Telemetría**
 - a. En cuanto al módulo ESP32, es un procesador de doble núcleo con gestión de radio Wi-Fi, cuya pila de comunicaciones es inherentemente no determinista. Si el STM32 dependiera del ESP32 para tomar decisiones, la fluctuación en la latencia de red (jitter) podría provocar errores de sincronización en procesos críticos.
 - b. **Decisión:** Restarle responsabilidad, el STM32 será el *maestro* del sistema, enviará una ráfaga de datos al terminar la recogida y el procesamiento de datos. El ESP32 solo recibirá los datos y se encargará de enviar los datos en orden hacia el servidor, manteniendo la integridad de estos.
2. **Riesgo de Inestabilidad por Gestión de Bus**
 - a. El uso del bus I2C para la lectura del MPU6050 a altas frecuencias (necesarias para el análisis de vibraciones) puede generar cuellos de botella y saturación en el bus, comprometiendo la periodicidad de los datos.

- b. **Decisión:** establecer en el flujo del sistema ventanas de 64 muestras, estableciendo delays en el código para la recogida de datos estable, evitando así la saturación de BUS.

Requisitos Software

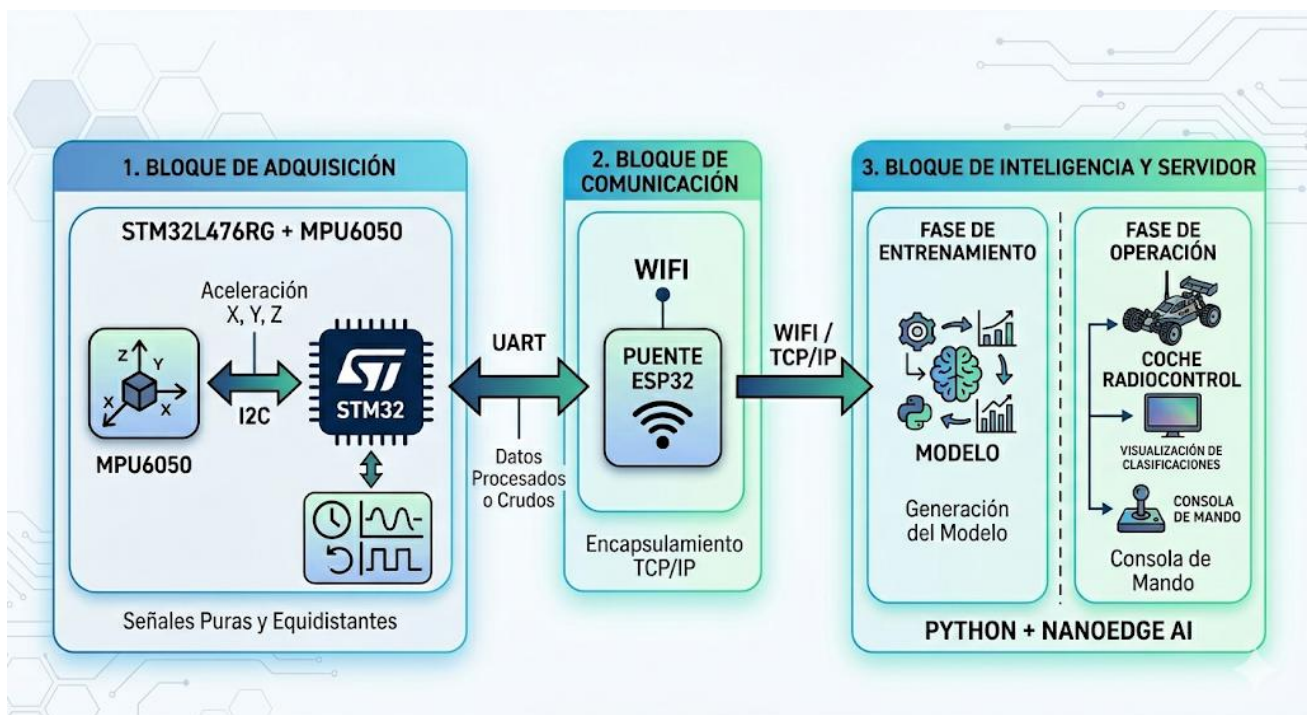
1. **NanoEdge AI Studio:** Entorno de desarrollo para la generación de la librería de inferencia (modelo). Es el núcleo software del proyecto para la detección de anomalías y clasificación
2. **STM32CubeIDE:** IDE oficial para el desarrollo del firmware en C/C++. Es necesario para la configuración de periféricos (DMA, I2C, UART) y la depuración del código en la arquitectura STM32L4.
3. **Visual Studio Code:** Entorno de desarrollo para el *backend* de visualización y el firmware del módulo ESP32 (usando el framework ESP-IDF o Arduino-ESP32).

BLOQUE 2: EJECUCIÓN DEL PROYECTO

B2.1. Diseño del Sistema

Esquema General y Diagrama de Bloques

- Bloque de Adquisición (STM32L476RG + MPU6050):** Este es el núcleo físico. El sensor MPU6050 capta las aceleraciones en los ejes X, Y y Z. Estos datos son leídos por el STM32 mediante el protocolo I2C. La funcionalidad de este bloque es garantizar que las señales lleguen de forma pura y equidistante en el tiempo al algoritmo de IA.
- Bloque de Comunicación (Puente ESP32):** Actúa como un intermediario transparente. Recibe los datos procesados o crudos por UART y los encapsula en paquetes TCP/IP para su transmisión vía WiFi. Su importancia radica en desacoplar las tareas de red de las tareas de tiempo real del STM32.
- Bloque de Inteligencia y Servidor (Python + NanoEdge AI):** En la fase de entrenamiento, este bloque recibe los datos crudos y genera el modelo. En la fase de operación, puede actuar como una consola de mando para visualizar las clasificaciones que el coche RC realiza localmente.



B2.2. Implementación

Tecnologías Empleadas

Hardware

- STM32 NUCLEO-L476RG

- El microcontrolador elegido pertenece a la serie de ultra-bajo consumo de ST. Basado en el núcleo ARM Cortex-M4, incluye una FPU que es vital para las operaciones de punto flotante de los modelos de IA.
- **Frecuencia:** 80 MHz.
- **Memoria** 1 MB Flash / 128 KB RAM.
- **Periféricos:** I2C1 para el sensor (Integrado), UART1 para comunicación con ESP32, UART2 para depuración USB.
- Basado en Cortex-M4 con FPU, 80 MHz, 1 MB Flash / 128 KB RAM.
- **MPU6050**
 - Acelerómetro de 6 ejes configurado en $\pm 2g$.
- **ESP-WROOM-32**
 - Destinada a puente UART-Wifi entre el microcontrolador STM32 y el host receptor de los datos (PC o similar)



Software

- **NanoEdge AI Studio**
 - Software de STM Electronics destinado a la IA embebida, concretamente a la detección de anomalías en sistemas y mantenimiento predictivo, esta será la herramienta matriz del proyecto, ya que permitirá implementar en el sistema el algoritmo de IA necesario para nuestro objetivo
 - Cuenta con AutoML integrada, la cual se encargará de proporcionar el algoritmo con mejor puntuación según los datos de entrada.
- **Entornos de desarrollo y lenguajes**
 - Visual Studio Code (extensión de platform.ini) para implementación de puerto wifi receptor de datos y generador de csv
 - STMCubeIDE para desarrollo del programa “DataLogger” encargado de recoger los datos proporcionados por el acelerómetro y enviarlos por la UART al ESP32

Desarrollo

Fase 1: Capa de Adquisición (Proyecto STM32) – Data logger

En esta fase nos encontramos con una de las partes más importantes para que la implementación sea exitosa y la IA sepa interpretar bien los datos, como sabemos, la IA necesita de una cantidad considerable de datos para funcionar correctamente y “comparar” y “predecir” estados, por ello cada

línea del dataset que va a recibir debe tener una cantidad suficiente de datos a interpretar. He optado por recoger los datos del acelerómetro en ventanas de 64 muestras (1 muestra: X, Y, Z), de esta forma cada línea de nuestro dataset tendrá 192 columnas, en un principio, suficientes para que la herramienta NanoEdge pueda trabajar con determinación

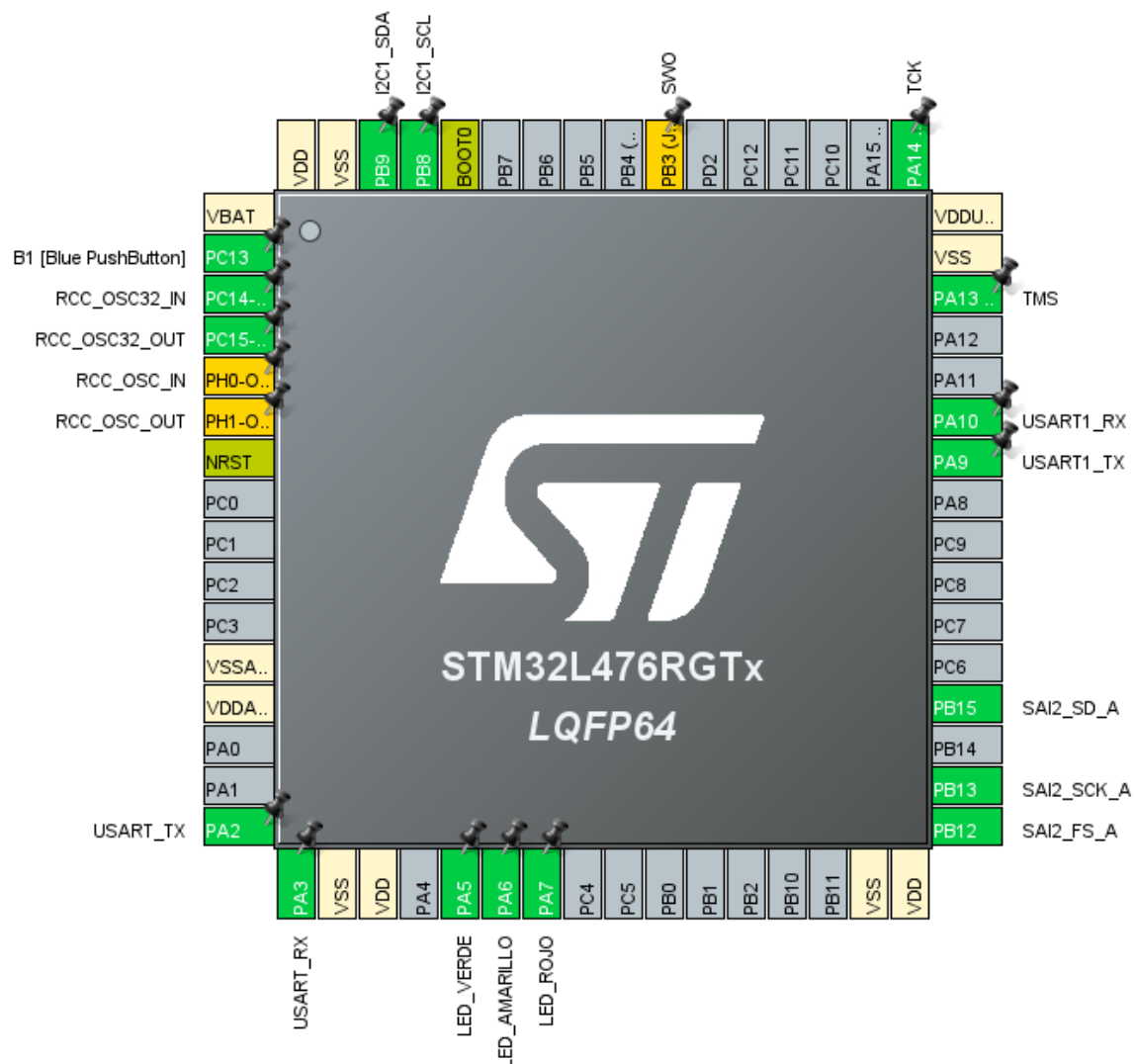
El desarrollo se realizó en C sobre el entorno STM32CubeIDE. El mayor reto fue asegurar el determinismo. Se implementó una lógica de "ventaneo":

Se capturan 64 muestras consecutivas por cada eje (X, Y, Z).

Esto genera una ventana de 192 valores ($64 * 3$).

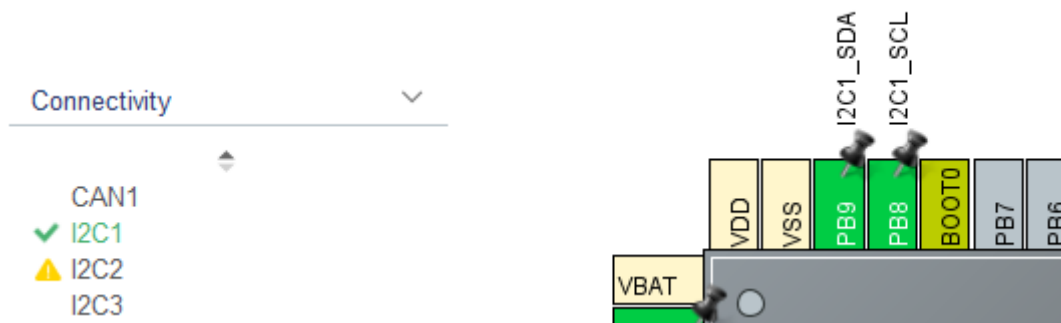
Se utiliza un retardo preciso de 10ms entre muestras para lograr 100Hz.

Configuración STMCubeIDE (DataLogger.ioc)



Visión General .ioc

Para poder recibir los datos del acelerómetro correctamente contamos con el bus I2C en nuestro microcontrolador NUCLEO, lo activamos:



A nivel hardware conectamos los pines según la siguiente imagen:

Fase 2: Capa de Red (Puente ESP32)

El ESP32 se programó en C++ (Arduino Framework). Implementa un servidor de sockets que escucha en el puerto 8082. Su lógica es un bucle infinito de lectura UART y escritura en Socket TCP.

Problema solucionado: La latencia del WiFi a veces bloqueaba el flujo de datos. Se solucionó mediante el uso de buffers circulares y tiempos de espera no bloqueantes.

Fase 3: Capa de IA (Pipeline de Datos)

El proceso de entrenamiento siguió estos pasos:

1. **Captura de Clases:** "Normal" (vuelo estable), "Anormal" (vibración por hélice dañada), "Parado".
2. **Limpieza:** Uso de un script Python para asegurar que cada ventana tenga exactamente 192 columnas, eliminando tramas con ruido UART.
3. **Benchmark:** NanoEdge AI analizó más de 500 combinaciones de algoritmos, seleccionando XGBoost por su equilibrio entre precisión (98.96%) y ligereza (1.2KB RAM).

Se implementó una lógica de ventaneo capturando 64 muestras por eje (192 valores totales) a 100Hz. El ESP32 gestiona la telemetría mediante sockets TCP para evitar bloqueos en el hilo principal del STM32.

BLOQUE 3: PLANIFICACIÓN DEL PROYECTO

B3.2. Planificación Financiera Inicial

Factura de Materiales (BOM):

Concepto	Unidades	Precio Unitario	Total
STM32L476RG Nucleo-64	1	25 €	25 €
ESP32 DevKit V1	1	10 €	10 €
Sensor MPU6050	1	5 €	5 €
Varios	1	20 €	20 €
Total Materiales			60 €

VII. CONCLUSIONES Y TRABAJO FUTURO

Se ha validado que la IA compleja es viable en microcontroladores de baja potencia utilizando menos de 2KB de RAM. Como trabajo futuro, se propone la integración con controladores de vuelo comerciales y el uso de Online Learning.